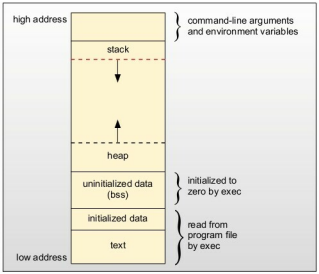

Figure 1: Memory organisation of a typical program code

Heap segment

When the program allocates memory at runtime using the calloc and malloc functions, then memory gets allocated in heap. When some more memory needs to be allocated using the calloc and malloc functions, heap grows upwards.

Memory management in C

There are two ways in which memory can be allocated in C:

- By declaring variables
- By explicitly requesting space from C

The C programming language provides several functions for memory allocation and management. These functions can be found in the <*stdlib.h*> header file.

All the functions are given below:

- *void *calloc(int num, int size):* This function allocates an array of *num* elements. The size of each element in bytes will be *size*.
- *void free(void *address):* This function releases a memory block specified by an address.
- *void *malloc(int num):* This function allocates an array of *num* bytes and leaves them initialised.
- *void *realloc(void *address, int newsize):* This function reallocates memory extending it up to *newsize*.

Allocating memory dynamically

While programming, if we are aware of the size of an array, it is easy to define the array. For example, to store the name of any person, if the array can go up to a maximum of 100 characters we can define something as follows:

```
char name[100];
```

But now, let us consider a situation where we have no idea about the length of the text we need to store. For example, if we want to store a detailed description about a topic, we need to define a pointer to a character without defining how much memory is required and later, based on the requirement, we can allocate memory, as shown in the example below:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main()
{
char name[100];
char * description;
strcpy(name, "Vikas Awasthy");
/* allocate memory dynamically * /
description = malloc( 200 * sizeof(char) );
if( description == NULL )
{
fprintf(stderr, "Error - unable to allocate required memory
\n");
}


else
{
strcpy( description, "Vikas Awasthy is an Application
Developer");
}
printf("Name = %s\n", name );
printf("Description: %s\n", description );
}

Output of Code:-

Name = Vikas Awasthy
Description: Vikas Awasthy is an Application Developer
```

Resizing and releasing memory

When our program is executed, the operating system automatically releases all the memory allocated by our program but as a good practice, when we are not in need of memory any more, then we should release that memory by calling the function *free()*.

Alternatively, we can increase or decrease the size of an allocated memory block by calling the function *realloc()*. Let us check the above program once again and make use of the *realloc()* and *free()* functions:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main()
{
char name[100];
char * description;
```